

- 1 Введение
- 2 Обзор литературы
- 3 Данные и предобработка
- 4 Качество инструкций: общий анализ
- 5 Гипотеза 1: Сложность определенной группы заданий
- 6 Гипотеза 2: Межпредметная согласованность паттернов ошибок
- 7 Аналитика после доработки датасета
- 8 Библиография

Создание синтетического инструктивного датасета по таксономии Блума: способности ЛЛМ генерировать задания разных типов

Noobel Alyona, Poletaev Nikita

2026-06-22

1 Введение

(ссылка на репозиторий: <https://github.com/solrpink/Creating-a-synthetic-instructional-dataset-LLM-s-ability-to-generate-different-types-of-tasks> (<https://github.com/solrpink/Creating-a-synthetic-instructional-dataset-LLM-s-ability-to-generate-different-types-of-tasks>))

Instruction tuning (IT), реализуемая преимущественно посредством контролируемого дообучения (supervised fine-tuning, SFT), превратилась в один из ключевых подходов к адаптации больших языковых моделей (LLM) для решения прикладных задач. В отличие от стандартного предобучения, которое оптимизирует модель для предсказания следующего токена, инструктивная настройка обучает LLM на парах «инструкция - ответ», обеспечивая способность следовать командам на естественном языке, отвечать на вопросы и решать структурированные задачи в широком диапазоне доменов. Данная парадигма особенно важна для образовательных приложений, где модель должна не просто генерировать связный текст, но и точно реагировать на запросы, адаптировать объяснения к уровню учащегося и поддерживать различные типы когнитивных операций.

1.1 Датасет и его основа

Настоящая работа опирается на синтетический инструктивный датасет, созданный по мотивам англоязычного датасета, описанного в статье **Lee, Bruce W., Cho, Hyunsoo, and Yoo, Kang Min — «Instruction Tuning with Human Curriculum»** [1]. Ключевая идея этой работы состоит в том, что порядок подачи обучающих примеров имеет принципиальное значение: обучение модели с нарастающей когнитивной сложностью (от простых заданий к сложным) позволяет достичь лучших результатов по сравнению с произвольным или фиксированным порядком. Авторы предложили структурировать инструктивные данные в соответствии с принципами человеческой педагогики, в частности, прогрессией от заданий на воспроизведение и понимание к заданиям на применение, анализ и синтез. Именно эта идея легла в основу структуры нашего датасета, организованного по таксономии Блума.

Датасет охватывает четыре научных домена (биология, химия, физика, география) на двух уровнях сложности. Каждый сгенерированный пример аннотирован согласно пересмотренной таксономии Блума [2], охватывающей три когнитивных уровня: **Запомнить**, **Понять** и **Применить**. Такая структура предназначена для обучения моделей не только предметным знаниям, но и образовательно значимым формам рассуждения и оценивания.

1.2 Цели исследования

Данная работа посвящена анализу результатов многоэтапной фильтрации сгенерированных инструкций с помощью подхода LLM-as-a-Judge. Мы исследуем:

- каков общий уровень качества инструкций в датасете;
- зависит ли качество от предметного домена;
- зависит ли качество от типа задания по таксономии Блума;
- является ли паттерн проблемных типов универсальным или доменно-специфичным.

2 Обзор литературы

2.1 Генерация учебных заданий с помощью ЛЛМ

Вопрос о том, способны ли LLM надёжно генерировать образовательные задания, выровненные по таксономии Блума, активно изучается в последние годы. Ниже представлен сравнительный обзор наиболее близких к нашей работе исследований.

2.1.1 Генерация школьных заданий с помощью ЛЛМ (ScienceDirect, 2025)

Работа, опубликованная в *Computers and Education: Artificial Intelligence* [3], исследует потенциал ЛЛМ (GPT-4 Turbo, GPT-3.5 Turbo, Llama-2-70B, Llama-3.1-405B, Gemini Pro) для автоматической генерации вопросов на основе учебников школьного уровня. Авторы применяли техники zero-shot и eight-shot промптинга и проводили ручную оценку по критериям охвата, грамматичности, полезности, отвечаемости и релевантности. Результаты показали, что eight-shot промптинг улучшает метрики оценки и способствует лучшему соответствию вопросов таксономии Блума.

Сравнение с нашей работой. Указанная работа опирается на ручную экспертную оценку, в нашей работе, напротив, используется автоматическая фильтрация через LLM-as-a-Judge, что позволяет обрабатывать значительно большие объёмы данных. Кроме того, мы вводим детальную типологию из 19 видов заданий по Блуму и сосредотачиваемся на анализе паттернов ошибок по типам, а не только на общей оценке качества.

2.1.2 Дополнительные смежные работы

Chen, S et al. [4] показали, что существующие бенчмарки для оценки ЛЛМ преимущественно ориентированы на нижние уровни таксономии Блума (Запомнить, Понять) и слабо представляют высшие уровни (Применить, Оценить, Создать). Это согласуется с нашими наблюдениями, задания на структурные связи, требующие более сложных когнитивных операций, демонстрируют более высокую долю ошибок генерации.

2.2 Что нового в нашей работе

В отличие от перечисленных исследований, настоящая работа:

1. **Вводит детальную типологию** из 19 (позже 22) конкретных видов заданий по Блуму, а не ограничивается шестью уровнями таксономии;
2. **Применяет автоматическую многокритериальную фильтрацию** (отвечаемость, фактологическая точность, утечка ответа) через LLM-as-a-Judge;
3. **Систематически исследует межпредметную воспроизводимость** паттернов ошибок с применением методов ранговой корреляции и байесовского анализа;
4. **Идентифицирует специфическую структурную проблему:** задания на сопоставление, аналогии и классификацию труднее для модели не из-за тематики, а из-за самого формата.

3 Данные и предобработка

Представленный датасет `all_subjects.csv` содержит 3988 строк и 14 колонок. Данные по 3-ем доменам (биология, химия, физика) сгенерированы синтетически с использованием языковой модели Qwen3-32b в рамках курсового группового проекта по созданию синтетического инструктивного датасета для fine tuning. Исходный материал: инструкции по предметам естественно-научного цикла, которые прошли многоэтапную автоматизированную валидацию по трём критериям: отвечаемость, фактологическая точность и отсутствие подсказок в формулировке вопроса. Результаты проверки включены в датасет в виде отдельных переменных.

3.1 Загрузка и первичный осмотр данных

```
# Исходная ссылка:
# https://drive.google.com/file/d/1yt2ESouGmjbpDU-ILHL7tc9ELQDbnYor/view?usp=sharing
data_url <- "https://drive.google.com/uc?export=download&id=1yt2ESouGmjbpDU-ILHL7tc9ELQDbnYor"
library(tidyverse)
library(showtext)

df <- read_csv(data_url, na = c("-", "-99"))

showtext_auto()
theme_set(
  theme_minimal(base_size = 13) +
    theme(
      plot.title = element_text(face = "bold", size = 16),
      plot.subtitle = element_text(size = 11),
      axis.title = element_text(face = "bold"),
      panel.grid.minor = element_blank(),
      legend.position = "bottom"
    )
)

df <- df %>%
  rename(
    grade = `класс`,
    answer = `Ответ`,
    question = `Вопрос`
  )
```

3.2 Структура датасета

```
# Структура
glimpse(df)
```

```
## Rows: 3,988
## Columns: 14
## $ grade          <chr> "7 класс", "8 класс", "9 класс", "10 класс", ...
## $ question       <chr> "Даны три примера: падение книги с полки, гор...
## $ answer         <chr> "Горение свечи относится к химическим явления...
## $ bloom_id       <dbl> 12, 14, 8, 7, 6, 14, 9, 5, 4, 16, 14, 17, 13,...
## $ answerability_score <dbl> 3, 4, 3, 3, 3, 4, 3, 3, 4, 3, 4, 3, 3, 4, 4, ...
## $ leakage_score   <dbl> 75, 15, 30, 35, 10, 10, 10, 10, 30, 10, 10, 1...
## $ accuracy_score  <dbl> 3, 4, 4, 4, 1, 4, 4, 1, 3, 3, 4, 4, 3, 3, NA,...
## $ answerability_verdict <chr> "Good", "Perfect", "Good", "Good", "Good", "P...
## $ hint_verdict    <chr> "FAIL", "PASS", "REVIEW", "REVIEW", "PASS", "...
## $ fact_verdict    <chr> "Mostly Correct", "Perfect", "Perfect", "Perf...
## $ answerability_issue_type <chr> "Minor Issues / Implicit Context", "", "Impli...
## $ hint_issue_type <chr> "MULTICHOICE_ELIMINATION", "OK", "LEADING HIN...
## $ fact_error_type <chr> "Minor Nitpick", "", "", "", "Hallucination",...
## $ Domain          <chr> "physics", "physics", "physics", "physics", "...

```

#видно, что в данных есть пропуски. Это произошло из-за ошибок генерации при работе с JSON

```
head(df)
```

```
## # A tibble: 6 × 14
##   grade    question          answer bloom_id answerability_score leakage_score
##   <chr>    <chr>          <chr>    <dbl>          <dbl>          <dbl>
## 1 7 класс  Даны три примера: ... Горен...      12              3              75
## 2 8 класс  Почему водяные кап... Водян...      14              4              15
## 3 9 класс  Даны следующие сит... Тело ...       8              3              30
## 4 10 класс Приведите пример ф... Приме...       7              3              35
## 5 11 класс Приведите пример, ... Для о...       6              3              10
## 6 7 класс  Почему звук не рас... Звук ...      14              4              10
## # i 8 more variables: accuracy_score <dbl>, answerability_verdict <chr>,
## #   hint_verdict <chr>, fact_verdict <chr>, answerability_issue_type <chr>,
## #   hint_issue_type <chr>, fact_error_type <chr>, Domain <chr>

```

```
# Размеры датасета
dim(df)
```

```
## [1] 3988   14
```

```
# Пропущенные значения
sum(is.na(df))
```

```
## [1] 186
```

```
colSums(is.na(df))
```

```
##          grade          question          answer
##          0              0              0
##      bloom_id  answerability_score  leakage_score
##          0              57              5
##      accuracy_score  answerability_verdict  hint_verdict
##      124              0              0
##      fact_verdict  answerability_issue_type  hint_issue_type
##          0              0              0
##      fact_error_type          Domain
##          0              0
```

*#мы видим, что пропуски есть именно в скорях
#но на самом деле, если есть пропуск, например, в answerability_score, то в answerability_i
ssue_type и answerability_verdict будет просто пустая строка
#это необходимо учитывать при работе с данными*

```
library(dplyr)
library(ggplot2)

n_examples <- 4

overview <- tibble(
  col = names(df),
  type = sapply(df, function(x) paste0("<", class(x)[1], ">")),
  examples = sapply(df, function(x) {
    vals <- na.omit(x)
    ex <- as.character(head(vals, n_examples))
    ex <- ifelse(nchar(ex) > 20, paste0(substr(ex, 1, 20), "..."), ex)
    # кавычки для chr, без для числовых и logical
    if (is.character(x)) ex <- paste0("'", ex, "'")
    paste(ex, collapse = ", ")
  })
) %>%
  mutate(y = rev(seq_len(n()))))
```

3.3 Описательные статистики

```
# Сводка по числовым колонкам
df %>%
  select(answerability_score, leakage_score, accuracy_score) %>%
  summary()
```

```
## answerability_score leakage_score accuracy_score
## Min. :0.000 Min. : 0.00 Min. :0.000
## 1st Qu.:3.000 1st Qu.:10.00 1st Qu.:3.000
## Median :4.000 Median :30.00 Median :3.000
## Mean :3.415 Mean :25.31 Mean :3.144
## 3rd Qu.:4.000 3rd Qu.:30.00 3rd Qu.:4.000
## Max. :4.000 Max. :90.00 Max. :4.000
## NA's :57 NA's :5 NA's :124
```

```
#видно, что более 75% данных во всех колонках оценены положительно
#для answerability_score и accuracy_score 1st Qu. = 3 (1 и 2 - отрицательные значения, 3 и
4 - положительные)
#для leakage_score 3rd Qu. = 30 (положительные значения <= 40)

# Детальные статистики
df %>%
  select(answerability_score, leakage_score, accuracy_score) %>%
  skimr::skim()
```

Data summary

Name	Piped data
Number of rows	3988
Number of columns	3
Column type frequency:	
numeric	3
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	po	p25	p50	p75	p100	hist
answerability_score	57	0.99	3.41	0.72	0	3	4	4	4	
leakage_score	5	1.00	25.31	19.44	0	10	30	30	90	
accuracy_score	124	0.97	3.14	1.05	0	3	3	4	4	

3.4 Распределение по доменам и классам

```
# Распределение по предметам и классам
df %>% count(Domain) %>% mutate(prop = n / sum(n) * 100)
```

```
## # A tibble: 3 × 3
##   Domain      n prop
##   <chr>    <int> <dbl>
## 1 biology  1425  35.7
## 2 chemistry 1139  28.6
## 3 physics  1424  35.7
```

```
df %>% count(grade) %>% mutate(prop = n / sum(n) * 100)
```

```
## # A tibble: 7 × 3
##   grade      n prop
##   <chr>    <int> <dbl>
## 1 10 класс   570 14.3
## 2 11 класс   569 14.3
## 3 5 класс   285  7.15
## 4 6 класс   285  7.15
## 5 7 класс   570 14.3
## 6 8 класс   855 21.4
## 7 9 класс   854 21.4
```

```
# Предмет × класс
```

```
df %>% count(Domain, grade) %>% pivot_wider(names_from = grade, values_from = n, values_fill = 0)
```

```
## # A tibble: 3 × 8
##   Domain `5 класс` `6 класс` `7 класс` `8 класс` `9 класс` `10 класс` `11 класс`
##   <chr>    <int>    <int>    <int>    <int>    <int>    <int>    <int>
## 1 biolo...    285     285     285     285     285         0         0
## 2 chemi...      0        0        0     285     285     285     284
## 3 physi...      0        0     285     285     284     285     285
```

```
# Распределение по вердиктам валидации
```

```
df %>% count(answerability_verdict)
```

```
## # A tibble: 6 × 2
##   answerability_verdict      n
##   <chr>                <int>
## 1 ""                     57
## 2 "Ambiguous"           278
## 3 "Deficient"           88
## 4 "Good"               1477
## 5 "Impossible"          1
## 6 "Perfect"            2087
```

```
df %>% count(hint_verdict)
```

```
## # A tibble: 4 × 2
##   hint_verdict      n
##   <chr>          <int>
## 1 ""              5
## 2 "FAIL"          389
## 3 "PASS"          1835
## 4 "REVIEW"        1759
```

```
df %>% count(fact_verdict)
```



```
## # A tibble: 7 × 2
##   fact_verdict      n
##   <chr>          <int>
## 1 ""             124
## 2 "False"        158
## 3 "Hallucination" 107
## 4 "Major Misconception" 155
## 5 "Mixed"        330
## 6 "Mostly Correct" 1282
## 7 "Perfect"      1832
```

```
# Распределение по типу ошибок
df %>% count(fact_error_type) %>% filter(n > 0)
```

```
## # A tibble: 15 × 2
##   fact_error_type      n
##   <chr>          <int>
## 1 ""             2196
## 2 "False"         1
## 3 "Hallucination" 209
## 4 "Incomplete"    1
## 5 "Incomplete Information" 4
## 6 "Logical Fallacy" 388
## 7 "Minor Nitpick" 538
## 8 "Mixed"         1
## 9 "Nitpick"       1
## 10 "Omission"      2
## 11 "Outdated Info" 6
## 12 "Significant Inaccuracy" 83
## 13 "Significant Omission" 1
## 14 "Terminological Error" 556
## 15 "Terminology"   1
```

```
df %>% count(hint_issue_type) %>% filter(n > 0)
```

```
## # A tibble: 8 × 2
##   hint_issue_type      n
##   <chr>          <int>
## 1 ""             5
## 2 "DIRECT_ANSWER" 29
## 3 "LEADING HINT" 1561
## 4 "LEADING_HINT" 122
## 5 "MULTICHOICE_ELIMINATION" 297
## 6 "OK"           1838
## 7 "OVERCONSTRAINED" 12
## 8 "REVEALING_EXAMPLE" 124
```

```
df %>% count(answerability_issue_type) %>% filter(n > 0)
```

```
## # A tibble: 9 × 2
##   answerability_issue_type      n
##   <chr>                    <int>
## 1 ""                      2144
## 2 "Ambiguous"              56
## 3 "Implicit Context"      1028
## 4 "Logical Error"         6
## 5 "Minor Issues"          1
## 6 "Minor Issues / Implicit Context" 323
## 7 "Missing Context"       37
## 8 "Missing Data"          377
## 9 "Subjective"            16
```

```
df <- df %>%
  mutate(
    answerability_verdict = if_else(is.na(answerability_verdict) | answerability_verdict ==
      "", "Unknown", answerability_verdict),
    hint_verdict = if_else(is.na(hint_verdict) | hint_verdict == "", "Unknown", hint_verdic
      t),
    fact_verdict = if_else(is.na(fact_verdict) | fact_verdict == "", "Unknown", fact_verdic
      t),
    Domain = if_else(is.na(Domain) | Domain == "", "Unknown", Domain)
  )

# Предмет × вердикт по фактологии
df %>% count(Domain, fact_verdict) %>%
  pivot_wider(names_from = fact_verdict, values_from = n)
```

```
## # A tibble: 3 × 8
##   Domain   False Hallucination `Major Misconception` Mixed `Mostly Correct`
##   <chr>     <int>         <int>             <int> <int>             <int>
## 1 biology      1           27                60  160             504
## 2 chemistry    24           33                54   86             373
## 3 physics    133           47                41   84             405
## # i 2 more variables: Perfect <int>, Unknown <int>
```

```
# по отвечаемости
df %>%
  count(Domain, answerability_verdict) %>%
  pivot_wider(names_from = answerability_verdict, values_from = n, values_fill = 0)
```

```
## # A tibble: 3 × 7
##   Domain   Ambiguous Deficient   Good Perfect Unknown Impossible
##   <chr>     <int>     <int> <int>  <int>  <int>    <int>
## 1 biology      98       31  529   766     1       0
## 2 chemistry    56       16  410   638    19       0
## 3 physics    124       41  538   683    37       1
```

```
# по подсказкам
df %>%
  count(Domain, hint_verdict) %>%
  pivot_wider(names_from = hint_verdict, values_from = n, values_fill = 0)
```

```
## # A tibble: 3 × 5
##   Domain      FAIL  PASS REVIEW Unknown
##   <chr>      <int> <int>  <int>    <int>
## 1 biology      121   626    677      1
## 2 chemistry    122   444    573      0
## 3 physics      146   765    509      4
```

Средние скоры по предметам

```
df %>%
  group_by(Domain) %>%
  summarise(
    mean_answerability = mean(answerability_score, na.rm = TRUE),
    mean_leakage = mean(leakage_score, na.rm = TRUE),
    mean_accuracy = mean(accuracy_score, na.rm = TRUE)
  )
```

```
## # A tibble: 3 × 4
##   Domain      mean_answerability mean_leakage mean_accuracy
##   <chr>                <dbl>         <dbl>         <dbl>
## 1 biology                3.43            25.5            3.22
## 2 chemistry              3.49            27.5            3.19
## 3 physics                3.34            23.4            3.03
```

скоры между предметами отличаются неявно

нет явного преимущества по качеству у одного из предметов

но позже мы посмотрим на графике, и ситуация станет яснее

4 Качество инструкций: общий анализ

4.1 Флаги качества и визуализация

```
# Флаги качества
df <- df %>%
  mutate(
    is_bad_answerability = answerability_score %in% c(1, 2),
    is_bad_accuracy      = accuracy_score %in% c(1, 2),
    is_bad_leakage       = leakage_score > 40,
    is_bad_any           = is_bad_answerability | is_bad_accuracy | is_bad_leakage,
    is_good              = !is_bad_any
  )
```

Доля хороших инструкций (по всем фильтрам)

```
df %>%
  summarise(total = n(),
            good = sum(is_good, na.rm = TRUE),
            prop_good = round(good / total * 100, 1)) %>%
  print()
```

```
## # A tibble: 1 × 3
##   total good prop_good
##   <int> <int>   <dbl>
## 1  3988  2727    68.4
```

```
# Доля хороших по доменам
```

```
df %>%
```

```
  group_by(Domain) %>%
```

```
  summarise(prop_good = mean(is_good, na.rm = TRUE) * 100) %>%
```

```
  ggplot(aes(x = Domain, y = prop_good, fill = Domain)) +
```

```
  geom_col() +
```

```
  geom_text(aes(label = paste0(round(prop_good, 1), "%")), vjust = -0.5, size = 3) +
```

```
  coord_cartesian(ylim = c(50, 80)) +
```

```
  scale_y_continuous(expand = expansion(mult = c(0, 0.05))) +
```

```
  scale_fill_manual(values = c("biology" = "#e41a1c",
                                "chemistry" = "#4daf4a",
                                "physics" = "#377eb8")) +
```

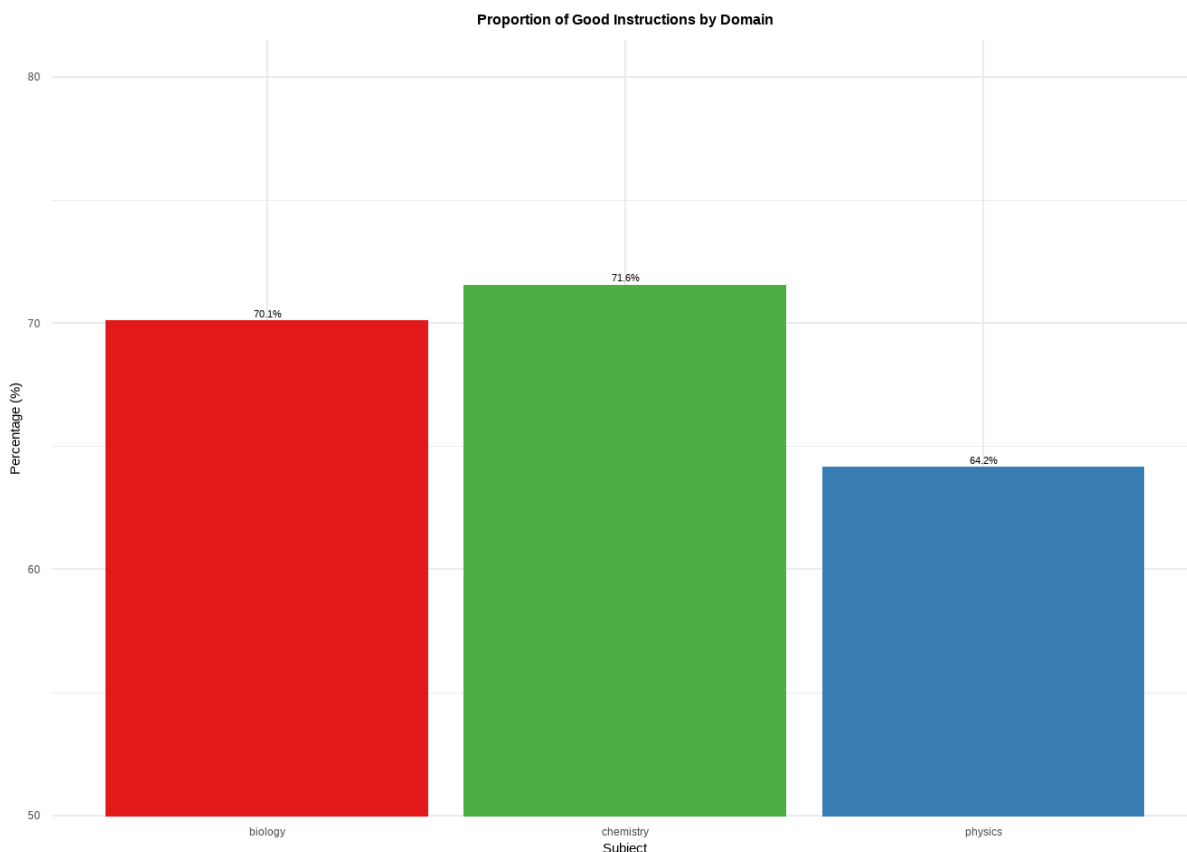
```
  labs(title = "Proportion of Good Instructions by Domain",
```

```
        x = "Subject", y = "Percentage (%)") +
```

```
  theme_minimal() +
```

```
  theme(legend.position = "none",
```

```
        plot.title = element_text(hjust = 0.5, size = 12, face = "bold"))
```



```
# Доля плохих по типам Блума (глобально)
```

```
df %>%
```

```
  group_by(bloom_id) %>%
```

```
  summarise(prop_bad = mean(is_bad_any, na.rm = TRUE) * 100) %>%
```

```
  ggplot(aes(x = reorder(bloom_id, prop_bad), y = prop_bad)) +
```

```
  geom_col(fill = "#4C78A8", width = 0.8) +
```

```
  geom_text(aes(label = sprintf("%.1f%%", prop_bad)),
```

```
            hjust = -0.15, size = 3, color = "grey30") +
```

```
  coord_flip(clip = "off") +
```

```
  scale_y_continuous(limits = c(0, 70), breaks = seq(0, 60, by = 20),
```

```
                    expand = expansion(mult = c(0, 0.05))) +
```

```
  labs(title = "Share of problematic instructions by Bloom category",
```

```
        x = "Bloom category", y = "% problematic") +
```

```
  theme_minimal(base_size = 12) +
```

```
  theme(
```

```
    plot.title       = element_text(face = "bold", size = 14, margin = margin(b = 12)),
```

```
    axis.title       = element_text(face = "bold"),
```

```
    axis.text        = element_text(color = "black", size = 11),
```

```
    axis.text.y      = element_text(face = "bold"),
```

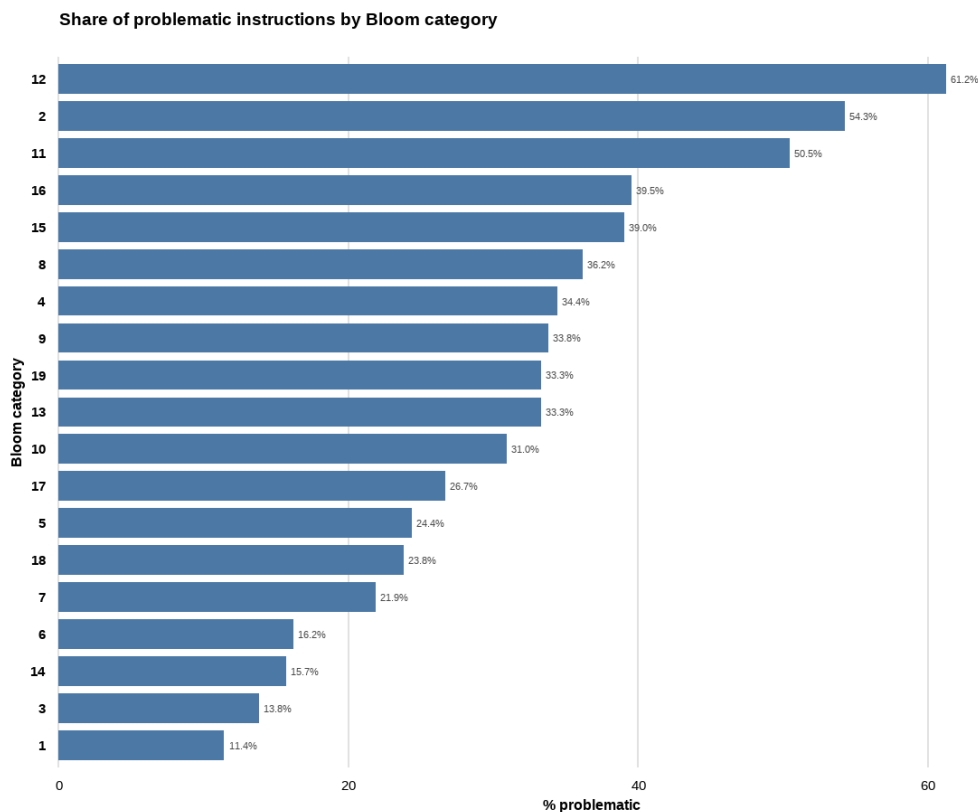
```
    panel.grid.major.y = element_blank(),
```

```
    panel.grid.minor  = element_blank(),
```

```
    panel.grid.major.x = element_line(color = "grey90", linewidth = 0.5),
```

```
    plot.margin       = margin(0.5, 1, 0.5, 0.5, "cm")
```

```
)
```



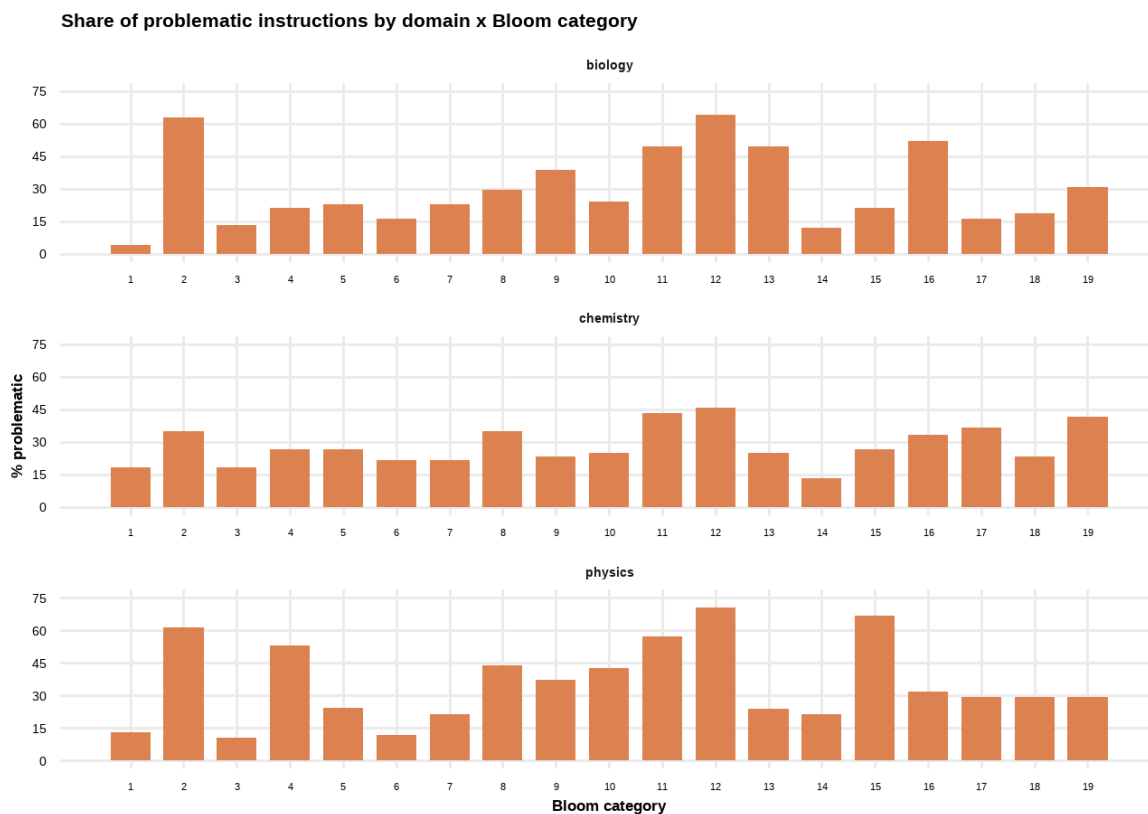
```

# Таблица: домен × Блум
df %>%
  group_by(Domain, bloom_id) %>%
  summarise(total = n(), bad_any = sum(is_bad_any, na.rm = TRUE),
            prop_bad_any = round(bad_any / total * 100, 1), .groups = "drop") %>%
  pivot_wider(names_from = Domain,
              values_from = c(total, bad_any, prop_bad_any), names_sep = "_") %>%
  View()

# Доля плохих инструкций по доменам × типам Блума
library(gggh4x)

df %>%
  mutate(bloom_id = as.numeric(as.character(bloom_id))) %>%
  group_by(Domain, bloom_id) %>%
  summarise(total = n(), bad_any = sum(is_bad_any, na.rm = TRUE),
            prop_bad = round(bad_any / total * 100, 1), .groups = "drop") %>%
  filter(total >= 5) %>%
  ggplot(aes(x = bloom_id, y = prop_bad)) +
  geom_col(fill = "#DD8452", width = 0.75) +
  facet_wrap2(~Domain, scales = "fixed", ncol = 1, axes = "all") +
  scale_y_continuous(limits = c(0, 75), breaks = seq(0, 75, by = 15)) +
  scale_x_continuous(breaks = 1:19) +
  labs(title = "Share of problematic instructions by domain x Bloom category",
       x = "Bloom category", y = "% problematic") +
  theme(
    strip.text = element_text(face = "bold"),
    axis.text = element_text(color = "black"),
    axis.text.x = element_text(angle = 0, size = 8),
    plot.margin = margin(0.5, 0.5, 0.5, 0.5, "cm")
  )

```



После подсчета статистик и ручной проверки статистически наиболее проблемных типов инструкций мы выявили недостатки фильтрации для типов 12 и 16.

Тип 16 содержит задания следующей структуры: «Задание на модификацию системы, в котором пользователю предлагается изменить систему для достижения определённой цели». Модель-судья расценивает многие подобные формулировки как неоднозначные, поскольку на такие вопросы существует несколько допустимых ответов, и по этой причине данные инструкции не проходят фильтр отвечаемости.

*При проверке результатов фильтрации типа 12 была обнаружена ошибка: в фильтр фактологической точности передавался только ответ без вопроса. Поэтому дальнейшей аналитике подвергнутся только **17 типов заданий**, 6 из которых относятся к структурным типам.*

5 Гипотеза 1: Сложность определенной группы заданий

Но: Доля некачественных инструкций статистически не различается между заданиями, требующими установления структурных связей (типы 2, 5, 7, 8, 11, 13), и иными заданиями.

Н1: Задания, требующие установления структурных связей, демонстрируют статистически значимо более высокую долю некачественных инструкций по сравнению с остальными типами заданий.

5.1 Описательные статистики по группам

```
# Исключаем типы 12 и 16 (ошибки фильтрации)
df_clean <- df %>%
  filter(!bloom_id %in% c(12, 16)) %>%
  mutate(bloom_id = as.integer(bloom_id))

# Группа "структурные связи": 2, 5, 7, 8, 11, 13 (сопоставление, аналогии, классификация)
structural_ids <- c(2, 5, 7, 8, 11, 13)

df_clean <- df_clean %>%
  mutate(
    task_group = if_else(bloom_id %in% structural_ids, "structural", "other")
  )

# Простые статистики по группам
group_summary <- df_clean %>%
  group_by(task_group) %>%
  summarise(
    n = n(),
    bad_n = sum(is_bad_any, na.rm = TRUE),
    prop_bad = mean(is_bad_any, na.rm = TRUE),
    se = sqrt(prop_bad * (1 - prop_bad) / n),
    ci_lo = prop_bad - 1.96 * se,
    ci_hi = prop_bad + 1.96 * se,
    mean_answer = mean(answerability_score, na.rm = TRUE),
    mean_accuracy = mean(accuracy_score, na.rm = TRUE),
    mean_leakage = mean(leakage_score, na.rm = TRUE)
  )

print(group_summary)
```

```
## # A tibble: 2 × 10
##   task_group      n bad_n prop_bad      se ci_lo ci_hi mean_answer mean_accuracy
##   <chr>      <int> <int>   <dbl>   <dbl> <dbl> <dbl>      <dbl>      <dbl>
## 1 other      2310   586   0.254 0.00905 0.236 0.272      3.41      3.27
## 2 structural 1259   463   0.368 0.0136 0.341 0.394      3.48      3.03
## # i 1 more variable: mean_leakage <dbl>
```

t-тест здесь неприменим, т.к. он требует, чтобы зависимая переменная была непрерывной и примерно нормально распределённой. У нас зависимая переменная `is_bad_any` принимает только два значения: 0 или 1 (бинарная). Здесь подойдет критерий хи-квадрат Пирсона + V Крамера (добавляет размер эффекта, которого одному p-значению не хватает).

5.2 Хи-квадрат Пирсона и V Крамера

```
# Таблица сопряжённости
contingency_table <- df_clean %>%
  mutate(quality = if_else(is_bad_any, "bad", "good")) %>%
  count(task_group, quality) %>%
  pivot_wider(names_from = quality, values_from = n, values_fill = 0) %>%
  column_to_rownames("task_group") %>%
  as.matrix()

print(contingency_table)
```

```
##           bad good NA
## other      586 1723  1
## structural 463  796  0
```

```
#хи-кв-тест Пирсона
chi2_result <- chisq.test(contingency_table)
print(chi2_result)
```

```
##
## Pearson's Chi-squared test
##
## data:  contingency_table
## X-squared = 51.531, df = 2, p-value = 6.459e-12
```

```
# Размер эффекта V Крамера
n_total <- sum(contingency_table)
cramers_v <- sqrt(chi2_result$statistic / (n_total * (min(nrow(contingency_table), ncol(contingency_table)) - 1)))
cat("Cramer's V:", round(cramers_v, 4), "\n")
```

```
## Cramer's V: 0.1202
```

5.3 Логистическая регрессия

```
# Логистическая регрессия: is_bad_any и task_group
# Посмотрим на размер эффекта в более понятных единицах (во сколько раз больше шанс получить плохую инструкцию в структурной группе)

logit_model <- glm(is_bad_any ~ task_group, data = df_clean, family = binomial)
summary(logit_model)
```

```
##
## Call:
## glm(formula = is_bad_any ~ task_group, family = binomial, data = df_clean)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.07850    0.04782 -22.553  < 2e-16 ***
## task_groupstructural  0.53663    0.07552   7.106 1.19e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 4322.2  on 3567  degrees of freedom
## Residual deviance: 4272.1  on 3566  degrees of freedom
## (1 пропущенное наблюдение удалено)
## AIC: 4276.1
##
## Number of Fisher Scoring iterations: 4
```

```
# Odds Ratio с 95% ДИ
exp_coef <- exp(cbind(OR = coef(logit_model), confint(logit_model)))
print(exp_coef)
```

```
##              OR      2.5 %    97.5 %
## (Intercept)   0.3401045 0.3094462 0.3732607
## task_groupstructural 1.7102342 1.4748183 1.9829992
```

5.4 Байесовский хи-квадрат

```
# Применим также байесовский хи квадрат для надежности выводов
library(BayesFactor)

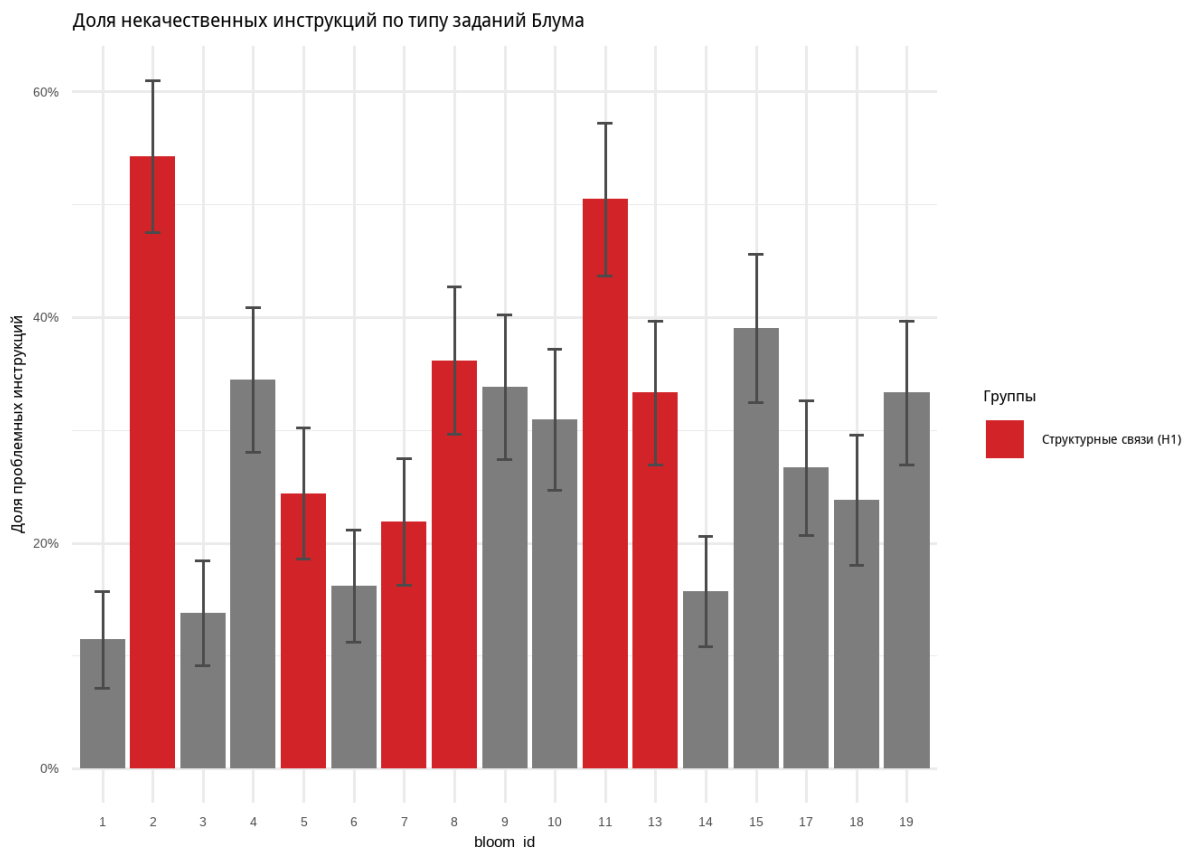
bf_result <- contingencyTableBF(contingency_table, sampleType = "indepMulti", fixedMargin =
  "rows")
print(bf_result)
```

```
## Bayes factor analysis
## -----
## [1] Non-indep. (a=1) : 11729421 ±0%
##
## Against denominator:
##   Null, independence, a = 1
## ---
## Bayes factor type: BFcontingencyTable, independent multinomial
```

5.5 Визуализация H1

```
# Визуализация для плохих инструкций по всем bloom_id с подсветкой групп
bloom_summary <- df_clean %>%
  group_by(bloom_id, task_group) %>%
  summarise(
    n          = n(),
    prop_bad   = mean(is_bad_any, na.rm = TRUE),
    se         = sqrt(prop_bad * (1 - prop_bad) / n),
    .groups    = "drop"
  )

ggplot(bloom_summary, aes(x = factor(bloom_id), y = prop_bad, fill = task_group)) +
  geom_col() +
  geom_errorbar(aes(ymin = prop_bad - 1.96 * se, ymax = prop_bad + 1.96 * se),
               width = 0.3, color = "gray30") +
  scale_fill_manual(
    values = c("structural" = "#D62728", "linear" = "#4C72B0"),
    labels = c("structural" = "Структурные связи (H1)", "linear" = "Другие")
  ) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(
    title = "Доля некачественных инструкций по типу заданий Блума",
    x      = "bloom_id",
    y      = "Доля проблемных инструкций",
    fill   = "Группы"
  ) +
  theme_minimal(base_size = 13)
```



Анализ:

Структурные задания демонстрируют более высокую долю некачественных инструкций: 36.8% против 25.4% у остальных. Интересно, что средний `answerability_score` у структурных даже выше (3.48 vs 3.41), то есть вопросы в них сформулированы понятно, но `accuracy_score` ниже (3.03 vs 3.27), а `leakage_score` значительно выше (31.2 vs 21.6). Это говорит о том, что проблема структурных заданий в том, что модель хуже генерирует фактически точные ответы и чаще проговаривает ответ в самом вопросе.

Хи-квадрат = 51.53, $p = 6.46 \times 10^{-12}$, что свидетельствует о статистической значимости отличия. V Крамера = 0.12 — эффект статистически значим, но малый.

По результатам логистической регрессии: $OR = 1.71 [1.47; 1.98]$, т.е. шансы получить некачественную инструкцию для структурного задания в 1.71 раза выше. $BF = 11\,729\,421$ — экстремально сильные свидетельства в пользу альтернативной гипотезы.

Вывод: Нулевая гипотеза отвергается. Задания на установление структурных связей (сопоставление, аналогии, классификация) действительно труднее для модели на уровне фактической точности ответа и утечки ответа в формулировку вопроса.

6 Гипотеза 2: Межпредметная согласованность паттернов ошибок

Н₀: Порядок типов заданий по доле некачественных инструкций не является согласованным между предметными доменами (биология, химия, физика).

Н₁: Порядок типов заданий по доле некачественных инструкций статистически значимо согласован между предметными доменами: типы с наибольшим числом ошибок воспроизводятся одинаково вне зависимости от тематики вопроса.

6.1 Ранги типов заданий по доменам

```
# Для каждого домена: ранг bloom_id по доле плохих инструкций
domain_bloom_ranks <- df_clean %>%
  group_by(Domain, bloom_id) %>%
  summarise(
    n          = n(),
    prop_bad = mean(is_bad_any, na.rm = TRUE),
    .groups   = "drop"
  ) %>%
  filter(Domain != "Unknown", n >= 5) %>%
  group_by(Domain) %>%
  mutate(rank_in_domain = rank(-prop_bad, ties.method = "average")) %>% # 1 = самый плохой
  ungroup()

print(n=51, domain_bloom_ranks)
```

```
## # A tibble: 51 × 5
##   Domain    bloom_id      n prop_bad rank_in_domain
##   <chr>      <int> <int>    <dbl>      <dbl>
## 1 biology         1    75    0.04         17
## 2 biology         2    75    0.627         1
## 3 biology         3    75    0.133        15
## 4 biology         4    75    0.213       10.5
## 5 biology         5    75    0.227         8.5
## 6 biology         6    75    0.16        13.5
## 7 biology         7    75    0.227         8.5
## 8 biology         8    75    0.293         6
## 9 biology         9    75    0.387         4
## 10 biology        10    75    0.24         7
## 11 biology        11    75    0.493         2.5
## 12 biology        13    75    0.493         2.5
## 13 biology        14    75    0.12         16
## 14 biology        15    75    0.213       10.5
## 15 biology        17    75    0.16       13.5
## 16 biology        18    75    0.187        12
## 17 biology        19    75    0.307         5
## 18 chemistry         1    60    0.183       15.5
## 19 chemistry         2    60    0.35         4.5
## 20 chemistry         3    60    0.183       15.5
## 21 chemistry         4    60    0.267         7
## 22 chemistry         5    60    0.267         7
## 23 chemistry         6    60    0.217       13.5
## 24 chemistry         7    60    0.217       13.5
## 25 chemistry         8    60    0.35         4.5
## 26 chemistry         9    60    0.233       11.5
## 27 chemistry        10    60    0.25         9.5
## 28 chemistry        11    60    0.433         1
## 29 chemistry        13    60    0.25         9.5
## 30 chemistry        14    60    0.133        17
## 31 chemistry        15    60    0.267         7
## 32 chemistry        17    60    0.367         3
## 33 chemistry        18    60    0.233       11.5
## 34 chemistry        19    60    0.417         2
## 35 physics          1    75    0.133        15
## 36 physics          2    75    0.613         2
## 37 physics          3    75    0.107        17
## 38 physics          4    75    0.541         4
## 39 physics          5    74    0.243        11
## 40 physics          6    75    0.12        16
## 41 physics          7    75    0.213       13.5
## 42 physics          8    75    0.44         5
## 43 physics          9    75    0.373         7
## 44 physics         10    75    0.427         6
## 45 physics         11    75    0.573         3
## 46 physics         13    75    0.24        12
## 47 physics         14    75    0.213       13.5
## 48 physics         15    75    0.667         1
## 49 physics         17    75    0.293         9
## 50 physics         18    75    0.293         9
## 51 physics         19    75    0.293         9
```

```
# Top-3 самых проблемных типа в каждом домене
top3_per_domain <- domain_bloom_ranks %>%
  group_by(Domain) %>%
  slice_min(rank_in_domain, n = 3) %>%
  select(Domain, bloom_id, prop_bad, rank_in_domain)

print(top3_per_domain)
```

```
## # A tibble: 9 × 4
## # Groups:   Domain [3]
##   Domain    bloom_id prop_bad rank_in_domain
##   <chr>      <int>   <dbl>         <dbl>
## 1 biology         2    0.627           1
## 2 biology        11    0.493          2.5
## 3 biology        13    0.493          2.5
## 4 chemistry       11    0.433           1
## 5 chemistry       19    0.417           2
## 6 chemistry       17    0.367           3
## 7 physics         15    0.667           1
## 8 physics          2    0.613           2
## 9 physics         11    0.573           3
```

6.2 Корреляции Спирмена между доменами

*Корреляция Спирмена выбрана потому, что мы сравниваем **ранговые** позиции типов заданий по доле ошибок в разных доменах. Спирмен работает с рангами, а не с числами напрямую. Мы берём её, потому что у нас нет оснований считать, что связь между доменами строго линейная и что доли ошибок нормально распределены. Спирмен мягче: он просто проверяет, что порядок типов по сложности примерно одинаковый в разных предметах.*

```
# Попарные корреляции Спирмена между доменами
domains_vec <- unique(domain_bloom_ranks$Domain[domain_bloom_ranks$Domain != "Unknown"])

# Матрица prop_bad: строки bloom_id, столбцы домены
prop_matrix <- domain_bloom_ranks %>%
  filter(Domain %in% domains_vec) %>%
  select(Domain, bloom_id, prop_bad) %>%
  pivot_wider(names_from = Domain, values_from = prop_bad) %>%
  column_to_rownames("bloom_id")

cat("\nКорреляция Спирмена между доменами (по доле плохих инструкций на тип задания)\n")
```

```
##  
## Корреляция Спирмена между доменами (по доле плохих инструкций на тип задания)
```

```
cor_matrix <- cor(prop_matrix, method = "spearman", use = "pairwise.complete.obs")  
print(round(cor_matrix, 3))
```

```
##           biology chemistry physics  
## biology    1.000    0.626  0.591  
## chemistry  0.626    1.000  0.725  
## physics    0.591    0.725  1.000
```

```
library(Hmisc)  
cor_test_matrix <- rcorr(as.matrix(prop_matrix), type = "spearman")  
cat("\np-values:\n")
```

```
##  
## p-values:
```

```
print(round(cor_test_matrix$P, 4))
```

```
##           biology chemistry physics  
## biology         NA    0.0072 0.0125  
## chemistry 0.0072         NA 0.0010  
## physics    0.0125 0.0010         NA
```

6.3 Байесовская корреляция

Это дополнение к обычному p -значению Спирмена. P -значение говорит только о том, есть ли связь, а Байес-фактор (BF) говорит, насколько данные поддерживают гипотезу о наличии корреляции по сравнению с её отсутствием. Например, $BF = 4$ означает: данные в 4 раза лучше объясняются тем, что корреляция есть, чем тем, что её нет.


```
# Байесовская корреляция для каждой пары доменов
library(BayesFactor)

domain_pairs <- combn(domains_vec, 2, simplify = FALSE)

for (pair in domain_pairs) {
  d1 <- pair[1]; d2 <- pair[2]

  x <- prop_matrix[, d1]
  y <- prop_matrix[, d2]
  complete_idx <- complete.cases(x, y)

  bf_corr <- correlationBF(x[complete_idx], y[complete_idx])
  cat(sprintf("\nБайес-фактор BF10 (%s vs %s): %.2f\n", d1, d2, extractBF(bf_corr)$bf))
}
```

```
##
## Байес-фактор BF10 (biology vs chemistry): 4.28
##
## Байес-фактор BF10 (biology vs physics): 3.78
##
## Байес-фактор BF10 (chemistry vs physics): 3.55
```

6.4 Тест Краскела–Уоллиса

Тест Краскела–Уоллиса применяется для проверки того, одинаковы ли медианные доли ошибок по типам заданий во всех трёх доменах: в отличие от ANOVA, он не требует нормальности распределения и работает с ранговыми данными.

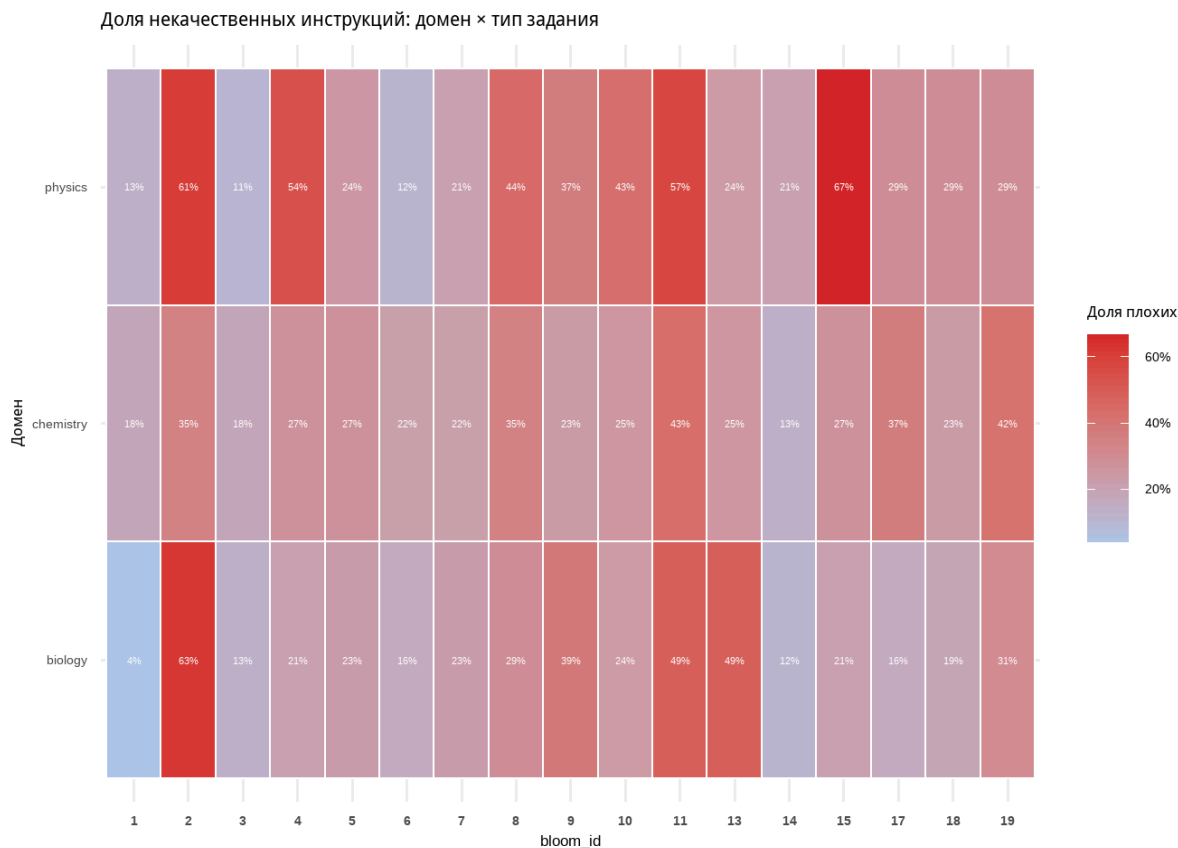
```
# Краскел-Уоллис: H0 – медианные доли ошибок по типам одинаковы во всех доменах
kw_result <- kruskal.test(prop_bad ~ Domain, data = domain_bloom_ranks)
print(kw_result)
```

```
##
## Kruskal-Wallis rank sum test
##
## data: prop_bad by Domain
## Kruskal-Wallis chi-squared = 2.1518, df = 2, p-value = 0.341
```

6.5 Тепловая карта: домен × тип задания

Тепловая карта

```
domain_bloom_ranks %>%
  ggplot(aes(x = factor(bloom_id), y = Domain, fill = prop_bad)) +
  geom_tile(color = "white", linewidth = 0.5) +
  geom_text(aes(label = sprintf("%.0f%%", prop_bad * 100)), size = 3, color = "white") +
  scale_fill_gradient(low = "#AEC6E8", high = "#D62728", labels = scales::percent_format())
  +
  labs(
    title = "Доля некачественных инструкций: домен × тип задания",
    x = "bloom_id",
    y = "Домен",
    fill = "Доля плохих"
  ) +
  theme_minimal(base_size = 13) +
  theme(axis.text.x = element_text(face = "bold"))
```



Анализ:

Все три пары доменов дают положительную и значимую корреляцию: все $p < 0.05$.

Наиболее близки chemistry и physics ($p = 0.725$). Биология несколько отличается ($p = 0.59 - 0.63$), что объяснимо: биология опирается больше на таксономию и описания, чем на формальные процедуры. Тем не менее ранговый порядок типов заданий по сложности для модели воспроизводится примерно одинаково вне зависимости от тематики.

BF для всех трёх пар находится в диапазоне 3.5–4.3 — умеренные свидетельства в пользу наличия корреляции.

Тест Краскела–Уоллиса: хи-квадрат = 2.15, $p = 0.341$ — не значим. Нет такого домена, который был бы систематически хуже или лучше остальных по всему спектру типов.

Вывод: Нулевая гипотеза отвергается. Тип 11 (анalogии) устойчиво входит в топ проблемных во всех доменах, тип 2 (сопоставление) — в двух из трёх. Задания на структурные связи труднее для модели не потому что она хуже знает какой-то предмет, а потому что сам формат задания предъявляет требования, с которыми модель справляется хуже вне зависимости от тематики.

7 Аналитика после доработки датасета

*Промежуточный анализ (Гипотезы 1 и 2) позволил выявить конкретные слабые места: ошибки фильтрации в типах 12 и 16, а также системно проблемные структурные типы заданий. На основе этих выводов был доработан датасет: исправлена передача контекста в фильтр фактологической точности, улучшен фильтр отвечаемости, введены новые типы заданий (2, 11, 20, 21, 22), которые являются аналогами прошлых проблемных типов 2 и 11 (это нужно, чтобы сложные задания были в достаточном количестве представлены в датасете после фильтрации, т.к. иначе большинство инструкция подобного формата просто отфильтруются). Ниже приведена аналитика **после** этих изменений — именно так промежуточный анализ помог нам улучшить итоговый результат.*

7.1 Загрузка обновлённых данных

```
# Обновлённый датасет после доработки:
# https://drive.google.com/file/d/1TabvfZM4vR-xEHumAwWr0ZDSK5Tlc35z/view?usp=drive_link
data_url_v2 <- "https://drive.usercontent.google.com/download?id=1TabvfZM4vR-xEHumAwWr0ZDSK5Tlc35z&export=download&authuser=0&confirm=t"

df2 <- read_csv(data_url_v2, na = c("-", "-99"))
```

7.2 Структура обновлённого датасета

```
# Структура
glimpse(df2)
```

```
## Rows: 38,865
## Columns: 23
## $ domain          <chr> "biology", "biology", "biology", "biology", "...
## $ grade           <chr> "5 класс", "5 класс", "5 класс", "5 класс", "...
## $ id              <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14...
## $ bloom_id        <dbl> 1, 5, 6, 8, 14, 18, 2, 7, 9, 15, 19, 22, 3, 1...
## $ definition       <chr> "биология – это наука о живых организмах и их...
## $ question        <chr> "Верно ли, что биология – это наука о живых о...
## $ answer          <chr> "верно", "Биология – это наука, изучающая орг...
## $ state           <chr> "[\"remember information\", \"easy\", \"ident...
## $ answerability_score <dbl> 4, 4, 3, 4, 4, 3, 4, 4, 4, 3, 3, 4, 4, 4, 3, ...
## $ answerability_verdict <chr> "Perfect", "Perfect", "Good", "Perfect", "Per...
## $ answerability_issue_type <chr> "", "", "Several Possible Answers", "", "", "...
## $ answerability_suggestion <chr> "OK.", "OK.", "Укажите конкретный организм, в...
## $ answerability_extras <chr> "{\"missing_elements\": \"None\", \"reasoning...
## $ accuracy_score    <dbl> 4, 4, 4, 3, 4, 4, 4, 3, 3, 4, 3, 4, 4, NA, 3,...
## $ accuracy_verdict  <chr> "Perfect", "Perfect", "Perfect", "Mostly Corr...
## $ accuracy_issue_type <chr> "", "", "", "Incomplete Answer", "", "", "", ...
## $ accuracy_suggestion <chr> "OK", "OK", "OK", "экология, зоология, физика...
## $ accuracy_extras   <chr> "{\"reasoning\": \"OK\", \"correction\": \"OK...
## $ leakage_score     <dbl> 0, 10, 10, 15, 10, 10, 10, 10, 10, 10, 10...
## $ hint_verdict      <chr> "PASS", "PASS", "PASS", "PASS", "PASS", "PASS...
## $ hint_issue_type   <chr> "OK", "OK", "OK", "OK", "OK", "OK", "OK", "OK...
## $ hint_suggestion   <chr> "[\"OK\"]", "[\"OK\"]", "[\"OK\"]", "[\"OK\"]...
## $ hint_extras       <chr> "{\"issues\": [{\"type\": \"OK\", \"severity\"...
```

```
head(df2)
```

```
## # A tibble: 6 × 23
##   domain grade      id bloom_id definition          question answer state
##   <chr>   <chr>   <dbl>   <dbl> <chr>          <chr>   <chr> <chr>
## 1 biology 5 класс     1       1 биология – это наука о ж... Верно л... верно "[\"...
## 2 biology 5 класс     2       5 биология – это наука о ж... Сформул... Биоло... "[\"...
## 3 biology 5 класс     3       6 биология – это наука о ж... Приведи... Приме... "[\"...
## 4 biology 5 класс     4       8 биология – это наука о ж... Какие и... эколо... "[\"...
## 5 biology 5 класс     5      14 биология – это наука о ж... Почему ... У жив... "[\"...
## 6 biology 5 класс     6      18 биология – это наука о ж... Как мож... Можно... "[\"...
## # i 15 more variables: answerability_score <dbl>, answerability_verdict <chr>,
## #   answerability_issue_type <chr>, answerability_suggestion <chr>,
## #   answerability_extras <chr>, accuracy_score <dbl>, accuracy_verdict <chr>,
## #   accuracy_issue_type <chr>, accuracy_suggestion <chr>,
## #   accuracy_extras <chr>, leakage_score <dbl>, hint_verdict <chr>,
## #   hint_issue_type <chr>, hint_suggestion <chr>, hint_extras <chr>
```

```
# Размеры датасета
dim(df2)
```

```
## [1] 38865    23
```

```
# Пропущенные значения
sum(is.na(df2))
```

```
## [1] 99
```

```
colSums(is.na(df2))
```

```
##           domain           grade           id
##           0             0             0
##      bloom_id      definition      question
##           0             0             0
##      answer        state  answerability_score
##           0             0             20
## answerability_verdict answerability_issue_type answerability_suggestion
##           0             0             0
## answerability_extras      accuracy_score      accuracy_verdict
##           0             59             0
## accuracy_issue_type      accuracy_suggestion      accuracy_extras
##           0             0             0
##      leakage_score      hint_verdict      hint_issue_type
##           20             0             0
##      hint_suggestion      hint_extras
##           0             0
```

```
library(dplyr)
library(ggplot2)

n_examples <- 4

overview2 <- tibble(
  col = names(df2),
  type = sapply(df2, function(x) paste0("<", class(x)[1], ">")),
  examples = sapply(df2, function(x) {
    vals <- na.omit(x)
    ex <- as.character(head(vals, n_examples))
    ex <- ifelse(nchar(ex) > 20, paste0(substr(ex, 1, 20), "..."), ex)
    if (is.character(x)) ex <- paste0("'", ex, "'")
    paste(ex, collapse = ", ")
  })
) %>%
  mutate(y = rev(seq_len(n())))
```

7.3 Описательные статистики

```
# Сводка по числовым колонкам
df2 %>%
  select(answerability_score, leakage_score, accuracy_score) %>%
  summary()
```

```
## answerability_score leakage_score accuracy_score
## Min. :0.000 Min. : 0.0 Min. :0.000
## 1st Qu.:3.000 1st Qu.: 10.0 1st Qu.:3.000
## Median :4.000 Median : 15.0 Median :4.000
## Mean :3.572 Mean : 21.3 Mean :3.431
## 3rd Qu.:4.000 3rd Qu.: 30.0 3rd Qu.:4.000
## Max. :4.000 Max. :100.0 Max. :4.000
## NA's :20 NA's :20 NA's :59
```

```
# Детальные статистики
df2 %>%
  select(answerability_score, leakage_score, accuracy_score) %>%
  skimr::skim()
```

Data summary

Name	Piped data
Number of rows	38865
Number of columns	3
Column type frequency:	
numeric	3
Group variables	None

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	po	p25	p50	p75	p100	hist
answerability_score	20	1	3.57	0.64	0	3	4	4	4	
leakage_score	20	1	21.30	17.84	0	10	15	30	100	
accuracy_score	59	1	3.43	0.87	0	3	4	4	4	

7.4 Распределение по доменам и классам

```
# Распределение по предметам и классам
df2 %>% count(domain) %>% mutate(prop = n / sum(n) * 100)
```

```
## # A tibble: 4 × 3
##   domain      n prop
##   <chr>    <int> <dbl>
## 1 biology  13554  34.9
## 2 chemistry 7280  18.7
## 3 geography 7045  18.1
## 4 physics  10986  28.3
```

```
df2 %>% count(grade) %>% mutate(prop = n / sum(n) * 100)
```

```
## # A tibble: 7 × 3
##   grade      n prop
##   <chr>    <int> <dbl>
## 1 10 класс 12125 31.2
## 2 11 класс 7631 19.6
## 3 5 класс 1497 3.85
## 4 6 класс 1740 4.48
## 5 7 класс 2974 7.65
## 6 8 класс 6613 17.0
## 7 9 класс 6285 16.2
```

7.5 Кросс-таблицы и средние скоры

```
df2 <- df2 %>%
  mutate(
    answerability_verdict = if_else(is.na(answerability_verdict) | answerability_verdict ==
      "", "Unknown", answerability_verdict),
    hint_verdict          = if_else(is.na(hint_verdict) | hint_verdict == "",
      "Unknown", hint_verdict),
    accuracy_verdict      = if_else(is.na(accuracy_verdict) | accuracy_verdict == "",
      "Unknown", accuracy_verdict),
    domain                = if_else(is.na(domain) | domain == "",
      "Unknown", domain)
  )

# Средние скоры по доменам
df2 %>%
  group_by(domain) %>%
  summarise(
    mean_answerability = mean(answerability_score, na.rm = TRUE),
    mean_leakage       = mean(leakage_score,      na.rm = TRUE),
    mean_accuracy      = mean(accuracy_score,    na.rm = TRUE)
  )
```

```
## # A tibble: 4 × 4
##   domain      mean_answerability mean_leakage mean_accuracy
##   <chr>                <dbl>         <dbl>         <dbl>
## 1 biology                3.55            21.1            3.46
## 2 chemistry               3.61            26.0            3.37
## 3 geography               3.51            23.1            3.34
## 4 physics                 3.62            17.2            3.50
```

7.6 Флаги качества и визуализация

```
# Флаги качества
df2 <- df2 %>%
  mutate(
    is_bad_answerability = answerability_score %in% c(1, 2),
    is_bad_accuracy       = accuracy_score %in% c(1, 2),
    is_bad_leakage        = leakage_score > 40,
    is_bad_any             = is_bad_answerability | is_bad_accuracy | is_bad_leakage,
    is_good                = !is_bad_any
  )

# Доля хороших инструкций (по всем фильтрам)
df2 %>%
  summarise(total      = n(),
             good       = sum(is_good, na.rm = TRUE),
             prop_good  = round(good / total * 100, 1)) %>%
  print()
```

```
## # A tibble: 1 × 3
##   total  good prop_good
##   <int> <int>   <dbl>
## 1 38865 31667    81.5
```

```
by_domain2 <- df2 %>%
  group_by(domain) %>%
  summarise(n = n(),
            prop_good = round(mean(is_good, na.rm = TRUE) * 100, 1)) %>%
  arrange(desc(prop_good))

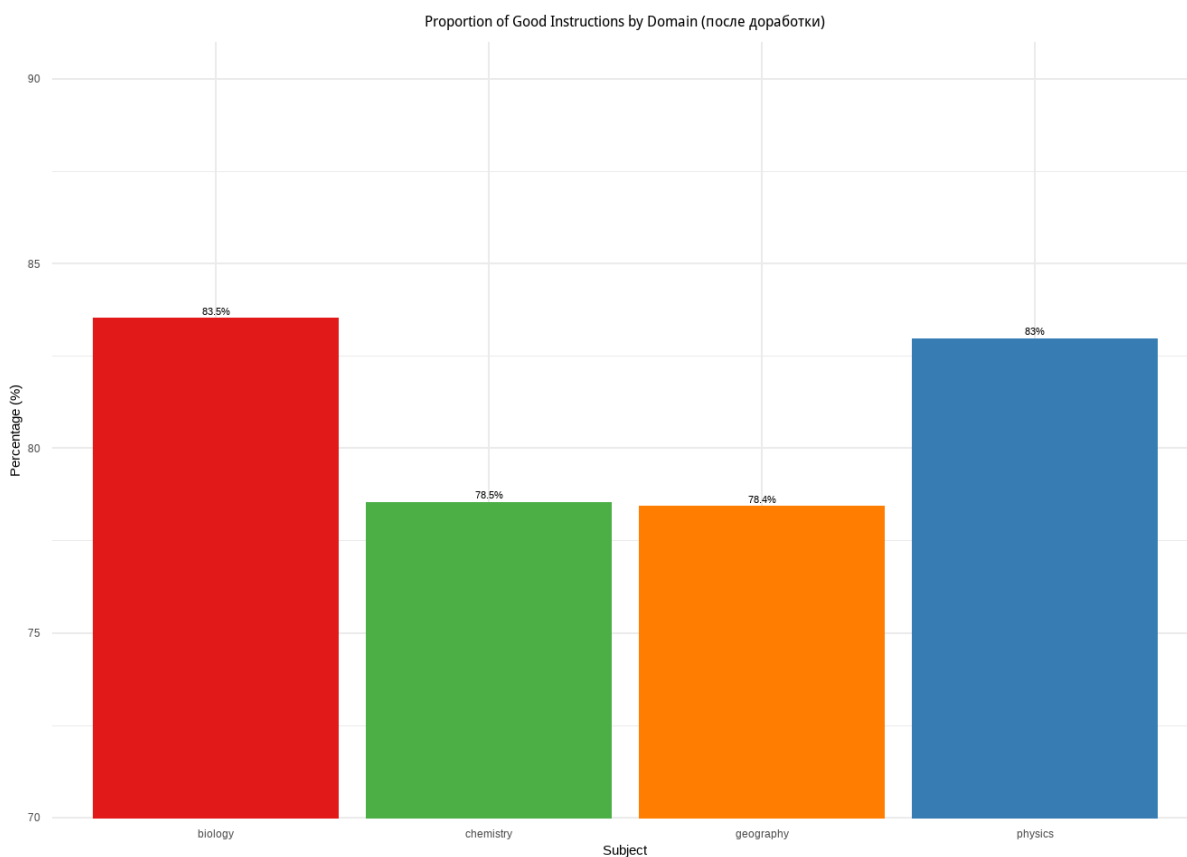
total_row2 <- df2 %>%
  summarise(domain = "TOTAL",
            n = n(),
            prop_good = round(mean(is_good, na.rm = TRUE) * 100, 1))

bind_rows(by_domain2, total_row2)
```

```
## # A tibble: 5 × 3
##   domain      n prop_good
##   <chr>   <int>   <dbl>
## 1 biology 13554    83.5
## 2 physics 10986     83
## 3 chemistry 7280    78.5
## 4 geography 7045    78.4
## 5 TOTAL   38865    81.5
```



```
# Доля хороших по доменам
df2 %>%
  group_by(domain) %>%
  summarise(prop_good = mean(is_good, na.rm = TRUE) * 100) %>%
  ggplot(aes(x = domain, y = prop_good, fill = domain)) +
  geom_col() +
  geom_text(aes(label = paste0(round(prop_good, 1), "%")), vjust = -0.5, size = 3) +
  coord_cartesian(ylim = c(70, 90)) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.05))) +
  scale_fill_manual(values = c("biology" = "#e41a1c",
                                "chemistry" = "#4daf4a",
                                "geography" = "#ff7f00",
                                "physics" = "#377eb8")) +
  labs(title = "Proportion of Good Instructions by Domain (после доработки)",
       x = "Subject", y = "Percentage (%)") +
  theme_minimal() +
  theme(legend.position = "none",
        plot.title = element_text(hjust = 0.5, size = 12, face = "bold"))
```



Доля плохих по типам Блума (глобально)

df2 %>%

group_by(bloom_id) %>%

summarise(prop_bad = mean(is_bad_any, na.rm = TRUE) * 100) %>%

ggplot(aes(x = reorder(bloom_id, prop_bad), y = prop_bad)) +

geom_col(fill = "#4C78A8", width = 0.8) +

geom_text(aes(label = sprintf("%.1f%%", prop_bad)),

hjust = -0.15, size = 3, color = "grey30") +

coord_flip(clip = "off") +

scale_y_continuous(limits = c(0, 70), breaks = seq(0, 40, by = 20),

expand = expansion(mult = c(0, 0.05))) +

labs(title = "Share of problematic instructions by Bloom category (после доработки)",

x = "Bloom category", y = "% problematic") +

theme_minimal(base_size = 9) +

theme(

plot.title = element_text(face = "bold", size = 12, margin = margin(b = 12)),

axis.title = element_text(face = "bold"),

axis.text = element_text(color = "black", size = 9),

axis.text.y = element_text(face = "bold"),

panel.grid.major.y = element_blank(),

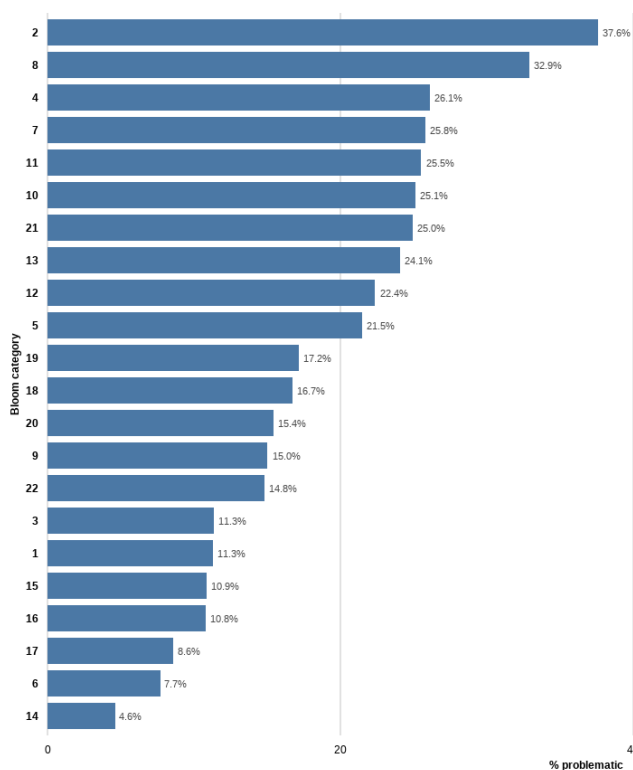
panel.grid.minor = element_blank(),

panel.grid.major.x = element_line(color = "grey90", linewidth = 0.5),

plot.margin = margin(0.5, 1, 0.5, 0.5, "cm")

)

Share of problematic instructions by Bloom category (после доработки)



```

# Таблица: домен × Блум
df2 %>%
  group_by(domain, bloom_id) %>%
  summarise(total = n(), bad_any = sum(is_bad_any, na.rm = TRUE),
            prop_bad_any = round(bad_any / total * 100, 1), .groups = "drop") %>%
  pivot_wider(names_from = domain,
              values_from = c(total, bad_any, prop_bad_any), names_sep = "_") %>%
  View()

# Доля плохих инструкций по доменам × типам Блума
library(ggh4x)

df2 %>%
  mutate(bloom_id = as.numeric(as.character(bloom_id))) %>%
  group_by(domain, bloom_id) %>%
  summarise(total = n(), bad_any = sum(is_bad_any, na.rm = TRUE),
            prop_bad = round(bad_any / total * 100, 1), .groups = "drop") %>%
  filter(total >= 5) %>%
  ggplot(aes(x = bloom_id, y = prop_bad)) +
  geom_col(fill = "#DD8452", width = 0.75) +
  facet_wrap2(~domain, scales = "fixed", ncol = 1, axes = "all") +
  scale_y_continuous(limits = c(0, 60), breaks = seq(0, 50, by = 20)) +
  scale_x_continuous(breaks = 1:22) +
  labs(title = "Share of problematic instructions by domain x Bloom category (после доработки)",
       x = "Bloom category", y = "% problematic") +
  theme(
    strip.text = element_text(face = "bold"),
    axis.text = element_text(color = "black"),
    axis.text.x = element_text(angle = 0, size = 8),
    plot.margin = margin(0.5, 0.5, 0.5, 0.5, "cm")
  )

```

Share of problematic instructions by domain x Bloom category (после доработки)



7.7 Описательные статистики по группам Н1 (обновлённый датасет)

```
df2_clean <- df2 %>%
  mutate(bloom_id = as.integer(bloom_id))

# Группа "структурные связи" – расширена с учётом новых типов 20, 21, 22
structural_ids_v2 <- c(2, 5, 7, 8, 11, 13, 20, 21, 22)

df2_clean <- df2_clean %>%
  mutate(
    task_group = if_else(bloom_id %in% structural_ids_v2, "structural", "other")
  )

# Простые статистики по группам
group_summary_v2 <- df2_clean %>%
  group_by(task_group) %>%
  summarise(
    n = n(),
    bad_n = sum(is_bad_any, na.rm = TRUE),
    prop_bad = mean(is_bad_any, na.rm = TRUE),
    se = sqrt(prop_bad * (1 - prop_bad) / n),
    ci_lo = prop_bad - 1.96 * se,
    ci_hi = prop_bad + 1.96 * se,
    mean_answer = mean(answerability_score, na.rm = TRUE),
    mean_accuracy = mean(accuracy_score, na.rm = TRUE),
    mean_leakage = mean(leakage_score, na.rm = TRUE)
  )

print(group_summary_v2)
```

```
## # A tibble: 2 × 10
##   task_group      n bad_n prop_bad      se ci_lo ci_hi mean_answer mean_accuracy
##   <chr>      <int> <int>   <dbl>   <dbl> <dbl> <dbl>      <dbl>      <dbl>
## 1 other      24359  3595   0.148 0.00227 0.143 0.152       3.50       3.49
## 2 structural 14506  3586   0.247 0.00358 0.240 0.254       3.7       3.33
## # i 1 more variable: mean_leakage <dbl>
```

7.8 Хи-квадрат Пирсона и V Крамера (обновлённый датасет)

```
# Таблица сопряжённости
contingency_table_v2 <- df2_clean %>%
  mutate(quality = if_else(is_bad_any, "bad", "good")) %>%
  count(task_group, quality) %>%
  pivot_wider(names_from = quality, values_from = n, values_fill = 0) %>%
  column_to_rownames("task_group") %>%
  as.matrix()

print(contingency_table_v2)
```

```
##          bad  good NA
## other      3595 20750 14
## structural 3586 10917  3
```

```
# ху-тест Пирсона
chi2_result_v2 <- chisq.test(contingency_table_v2)
print(chi2_result_v2)
```

```
##
## Pearson's Chi-squared test
##
## data:  contingency_table_v2
## X-squared = 601.11, df = 2, p-value < 2.2e-16
```

```
# Размер эффекта V Крамера
n_total_v2 <- sum(contingency_table_v2)
cramers_v_v2 <- sqrt(chi2_result_v2$statistic / (n_total_v2 * (min(nrow(contingency_table_v2), ncol(contingency_table_v2)) - 1)))
cat("Cramer's V:", round(cramers_v_v2, 4), "\n")
```

```
## Cramer's V: 0.1244
```

7.9 Логистическая регрессия (обновлённый датасет)

```
# Логистическая регрессия: is_bad_any u task_group
logit_model_v2 <- glm(is_bad_any ~ task_group, data = df2_clean, family = binomial)
summary(logit_model_v2)
```

```
##
## Call:
## glm(formula = is_bad_any ~ task_group, family = binomial, data = df2_clean)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -1.75300    0.01807  -97.04  <2e-16 ***
## task_groupstructural  0.63972    0.02640   24.23  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 37190  on 38847  degrees of freedom
## Residual deviance: 36607  on 38846  degrees of freedom
## (17 пропущенных наблюдений удалены)
## AIC: 36611
##
## Number of Fisher Scoring iterations: 4
```

```
# Odds Ratio c 95% ДИ
exp_coef_v2 <- exp(cbind(OR = coef(logit_model_v2), confint(logit_model_v2)))
print(exp_coef_v2)
```

```
##                OR      2.5 %    97.5 %
## (Intercept)    0.173253 0.1672011 0.1794711
## task_groupstructural 1.895947 1.8003558 1.9966344
```

7.10 Байесовский хи-квадрат (обновлённый датасет)

```
# Применим также байесовский хи квадрат для надёжности выводов
library(BayesFactor)

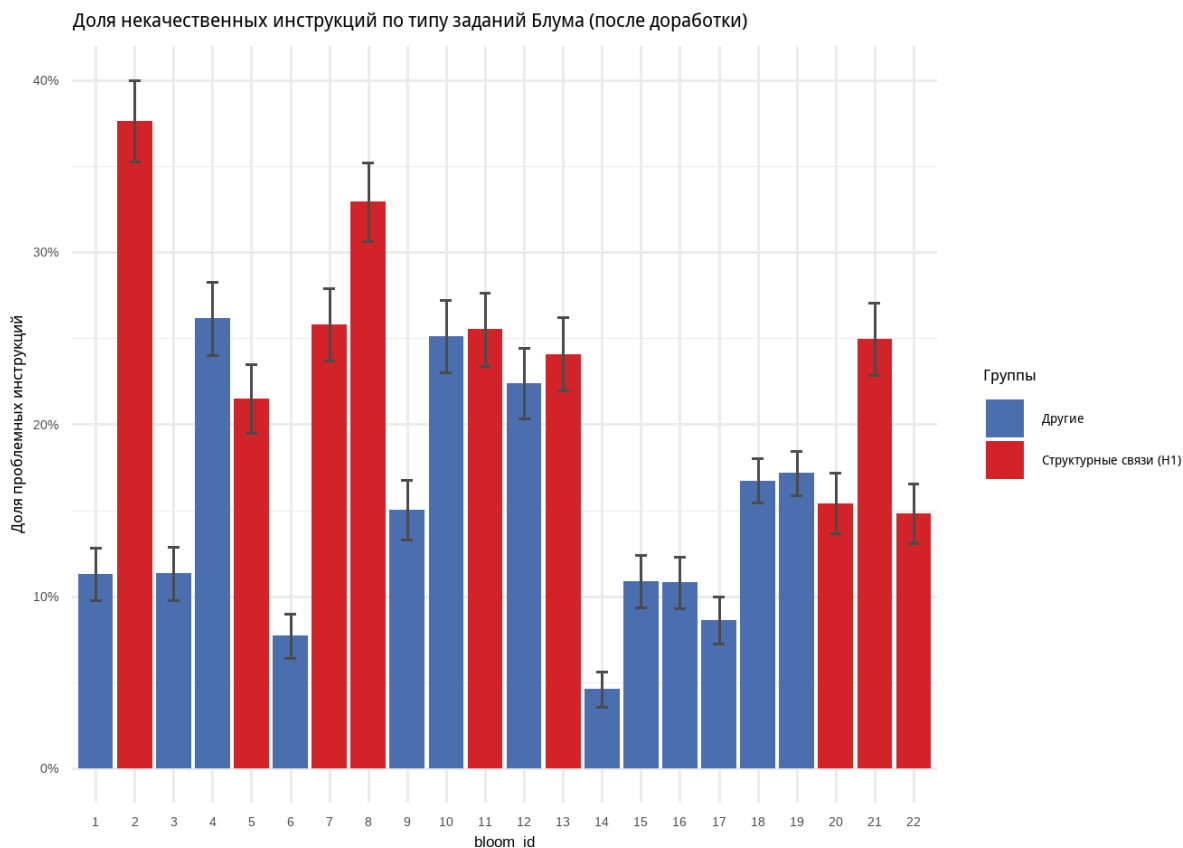
bf_result_v2 <- contingencyTableBF(contingency_table_v2, sampleType = "indepMulti", fixedMa
  rgin = "rows")
print(bf_result_v2)
```

```
## Bayes factor analysis
## -----
## [1] Non-indep. (a=1) : 2.295453e+122 ±0%
##
## Against denominator:
##   Null, independence, a = 1
## ---
## Bayes factor type: BFcontingencyTable, independent multinomial
```

7.11 Визуализация H1 (обновлённый датасет)

```
# Визуализация доли плохих инструкций по всем bloom_id с подсветкой групп
bloom_summary_v2 <- df2_clean %>%
  group_by(bloom_id, task_group) %>%
  summarise(
    n      = n(),
    prop_bad = mean(is_bad_any, na.rm = TRUE),
    se      = sqrt(prop_bad * (1 - prop_bad) / n),
    .groups = "drop"
  )

ggplot(bloom_summary_v2, aes(x = factor(bloom_id), y = prop_bad, fill = task_group)) +
  geom_col() +
  geom_errorbar(aes(ymin = prop_bad - 1.96 * se, ymax = prop_bad + 1.96 * se),
               width = 0.3, color = "gray30") +
  scale_fill_manual(
    values = c("structural" = "#D62728", "other" = "#4C72B0"),
    labels = c("structural" = "Структурные связи (H1)", "other" = "Другие")
  ) +
  scale_y_continuous(labels = scales::percent_format(accuracy = 1)) +
  labs(
    title = "Доля некачественных инструкций по типу заданий Блума (после доработки)",
    x      = "bloom_id",
    y      = "Доля проблемных инструкций",
    fill   = "Группы"
  ) +
  theme_minimal(base_size = 13)
```



Итоговый анализ структуры датасета после доработки:

Обновлённый датасет включает четыре домена (биология, химия, физика, география) и расширенный набор типов заданий (до 22). Общий уровень качества инструкций вырос: доля хороших инструкций около 80% по всем доменам, что заметно лучше предыдущих показателей. Качество датасета улучшилось на 13.1%.

Паттерн проблемных типов заданий сохраняется: структурные типы (сопоставление, аналогии, классификация) по-прежнему демонстрируют более высокую долю некачественных инструкций, чем остальные. Расширение группы структурных типов за счёт новых позиций (20, 21, 22) подтверждает универсальность этого эффекта — он не является артефактом первоначальной выборки типов.

Относительно небольшая модель Qwen3-32B значительно хуже справляется с генерацией структурных инструкций: доля проблемных выходов для структурных заданий составляет 24.7% против 14.8% для остальных типов. Это подкреплено экстремально сильными статистическими свидетельствами ($p < 2.2 \times 10^{-16}$, $BF = 2.29 \times 10^{122}$). Логистическая регрессия количественно описывает эту уязвимость: шансы получить некачественную структурную инструкцию почти вдвое выше, чем для других категорий ($OR = 1.90$, 95% ДИ [1.80; 1.99]).

Вывод: Промежуточный анализ дал конкретный практический результат: были устранены системные ошибки фильтрации, расширен охват доменов и типов заданий, а итоговый датасет стал надёжнее и репрезентативнее. При этом ключевой содержательный вывод — структурные задания труднее для модели вне зависимости от предметной области — подтверждается и на обновлённых данных.

8 Библиография

- [1] Lee, B. W., Cho, H., & Yoo, K. M. (2023). *Instruction Tuning with Human Curriculum*.
- [2] Anderson, L. W., et al. (2001). *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*.
- [3] Maity, S. et al. (2025). *Can large language models meet the challenge of generating school-level questions?*
- [4] Chen, S et al. (2025). *Automated Benchmark Generation from Domain Guidelines Informed by Bloom's Taxonomy*.